Capstone Two - Final Project Report: Predicting Wafer Failures from Process Sensors (SECOM)

Nov 9, 2026 · @Thomas

Executive summary

Process sensors carry a weak but real signal for wafer failure: the final Logistic Regression model scores a test PR AUC of 0.166, about three times the 0.054 random baseline, and catches 5 of the 17 failures in the latest test period.

The project uses the UCI SECOM dataset (1,567 units, 590 sensors, 6.6% failures). Data was split by time, cleaned using training data only, and three model families were tuned with time-ordered cross-validation. Logistic Regression with 50 selected features was the only model that held up on later wafers. LightGBM and Random Forest looked better in cross-validation but fell close to random on the test period and caught no failures.

The main caveat is precision: about 9 in 10 alerts are false alarms (precision 0.106), and the test set has only 17 failures. The model is a screening aid, not an automatic reject rule.

1. Problem and data

The goal is to predict, from in-line sensor readings, which wafers will fail final test, so that likely failures can be flagged before more cost is spent on them.

The data is the UCI SECOM dataset: 1,567 production units recorded from 2008-07-19 to 2008-10-17, each with 590 sensor readings, a timestamp and a pass/fail label. Only 104 units fail (6.6%), roughly 13 passes for every failure.

This imbalance shapes the whole project. A model that always predicts "pass" is about 93% accurate and finds nothing, so accuracy is not used. Models are judged mainly on PR AUC (average precision), which focuses on the rare failure class, with ROC AUC, recall and precision as supporting metrics.

2. Data wrangling

Cleaning cut the data from 590 to 436 sensors, added 28 missing-value flags, and removed no units. Notebook: `01_data_wrangling.ipynb`.

Two rules guided every step. First, the data was split by time before any statistic was computed: the earliest 80% (1,253 units, 87 failures, up to 2008-10-02) is training and the latest 20% (314 units, 17 failures) is test. A random split would train on the future and overstate performance. Second, medians and clipping limits are learned from training data only and applied unchanged to the test set.

Step	Result
Mostly-missing sensors (over 50% missing) dropped	24
Constant sensors dropped	122
Duplicated sensors (correlation above 0.9999) dropped	8
Missing values filled	median from training set; none left
Missing-value flags added (sensors missing over 2%)	28
Extreme values clipped to the 0.5% and 99.5% points	9.5% of training readings
Units removed	0
Final dataset	464 columns; 1,253 train and 314 test units

Units with extreme readings were clipped, not deleted, because with only 104 failures those readings are likely to be the failures. Scaling, feature selection and class balancing were left out of the saved data on purpose. They run inside the modelling pipeline so they are fitted separately for each cross-validation fold and cannot leak.

3. Exploratory analysis

No single sensor separates failures from passes: the best one reaches an AUC of only 0.728, and the median sensor sits just 0.024 from the no-signal value of 0.5. Notebook: `02_eda_secom.ipynb`, run on the training set only.

- **Skewed, uneven scales.** 118 of 436 sensors have absolute skew above 2, and standard deviations range from about 1e-4 to 6e3, so scaling must happen inside the model pipeline.
- **Weak but real signal.** Each sensor was tested with Mann-Whitney U (equivalent to per-sensor AUC) and Benjamini-Hochberg false-discovery correction. 20 of 436 sensors are significant after correction, and 70 pass an uncorrected 0.05 test against about 22 expected by chance. The strongest is S059 (AUC 0.728).
- **Failures are shifted and more spread out.** 72.4% of failures fall inside the central 90% range of two top sensors at once, so no simple two-sensor rule works.
- **Missing-value flags carry information.** Units with a missing flag fail more often (8.8% against 6.4%, a lift of 1.37), so the flags were kept.
- **Block correlation.** Most sensor pairs are unrelated (median absolute correlation 0.030), but 298 of 436 sensors have a partner above 0.9. These are probably sensors on the same tool or process step. This favours regularised or ensemble models over plain regression.

Taken together, the signal is weak and spread across many correlated sensors, which points to regularised models, time-ordered validation and PR AUC as the metric.

4. Modeling approach

Three model families were compared, all with class weights to handle the 13-to-1 imbalance. Notebook: `04_models.ipynb`.

Model	Type	Why chosen
Logistic Regression (L1, with feature selection)	Linear	Simple, interpretable baseline
Random Forest	Bagged trees	Handles many noisy features and non-linear effects
LightGBM	Gradient boosting	Strong on tabular data

Hyperparameters were tuned with `GridSearchCV` (Logistic Regression, LightGBM) or `RandomizedSearchCV` (Random Forest, 40 configurations), scored on PR AUC. Because the data is time-ordered, every search used `TimeSeriesSplit` with 5 folds and a gap of 10 units, so models always train on the past and validate on the future. The test set (the latest 20% of units) was used once, after tuning.

PR AUC is the main metric because a random model scores only about 0.054 (the failure rate). ROC AUC, recall and precision on the failure class are reported alongside it.

Earlier experiments (scripts `05` to `10` in the repository) also tried XGBoost, CatBoost, ExtraTrees, SVM, kNN and SMOTE variants. None gave a consistent improvement on forward-in-time validation.

5. Results

Logistic Regression is the final model because it is the only one whose cross-validation result held up on the later test period.

Model	Best settings	CV PR AUC	Test PR AUC	Test ROC AUC	Recall (fail)	Precision (fail)	F1
Logistic Regression	L1, C=1, top 50 features	0.182	**0.166**	0.574	0.294	0.106	0.156
LightGBM	100 trees, learning rate 0.03, 8 leaves	**0.247**	0.078	0.580	0.000	0.000	0.000
Random Forest	500 trees, depth 8, max_features 0.3	0.229	0.083	0.605	0.000	0.000	0.000
Random baseline	n/a	n/a	0.054	0.500	n/a	n/a	n/a

Metrics are on the 314-unit test set (17 failures) at the default 0.5 probability cutoff. The full table is in `outputs/model_metrics.csv`.

- **Cross-validation misled.** LightGBM and Random Forest scored best in cross-validation but fell to 0.078 and 0.083 on the test period, close to the 0.054 baseline, and neither caught a single failure. They appear to have learned patterns from the training period that do not carry forward in time.
- **Logistic Regression held up.** Its test PR AUC of 0.166 is about 3 times the baseline, and it flagged 5 of the 17 failures (recall 0.294).
- **ROC AUC is modest for every model** (0.57 to 0.61), only slightly above random, which confirms the signal is weak.
- **Accuracy is misleading.** LightGBM and Random Forest reach 93% and 95% accuracy by predicting almost everything as pass.

6. Limitations

The results are uncertain mainly because the test period contains only 17 failures.

- **Small test set.** One more or one fewer detected failure moves recall by about 6 points, so differences between models are noisy.
- **Selection rests on one test result.** Cross-validation preferred LightGBM, so choosing Logistic Regression relies on the single test outcome. It should be confirmed with more data or a bootstrap confidence interval.
- **Low precision.** Precision is 0.106, so about 9 in 10 alerts are false alarms.
- **Weak signal.** Test ROC AUC of 0.57 to 0.61 means the models only slightly rank failures above passes. Changing the probability cutoff did not fix this: even a cutoff chosen using the test labels reached an F1 of only about 0.2, so the limit is the signal and not the model setup.
- **Short time window.** The data covers three months, so drift over longer periods cannot be assessed.

7. Recommendations and next steps

Use the Logistic Regression model as a screening aid that prioritises wafers for extra inspection, not as an automatic reject rule, because most of its alerts are false alarms.

Next steps, in order of expected value:

1. Add bootstrap confidence intervals on test PR AUC, recall and precision to show how much of the result is chance.
2. Engineer instability features suggested by the EDA, such as rolling standard deviation, absolute z-score and lagged readings.
3. Evaluate with rolling-origin (walk-forward) validation over many time windows, which uses more of the 104 failures than a single split.
4. Calibrate predicted probabilities and weigh the cost of a missed failure against the cost of a false alarm when setting the alert level.
5. Collect more failure examples; the biggest limit on every model here is how few there are.

Sources

- McCann, M. and Johnston, A. (2008). SECOM, UCI Machine Learning Repository.
- Project notebooks and outputs: `01_data_wrangling.ipynb`, `02_eda_secom.ipynb`, `03_preprocessing.ipynb`, `04_models.ipynb`, `outputs/`.